

12.1 The Forward Process

Jump to x_t in one shot. α -bar from 1 to 0. The kernel is not learned.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

A **diffusion** model learns to denoise. The forward process is not learned. You add Gaussian noise on a schedule until the sample is almost prior noise. Training later asks a net to predict that noise. By the end you should be able to write α_t and $\bar{\alpha}_t$, jump to any t in one line, and say why that jump is nicer to train than a GAN min-max.

1. What the diffusion forward process is

Diffusion (DDPM) **destroys** a sample with Gaussian noise on a schedule, then later learns to reverse it. Start from data x_0 . A variance schedule $0 < \beta_t < 1$ (small at first, larger later) defines a Markov chain $x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_T$. At $t = 0$ you still have the data. At $t = T$ you have nearly isotropic noise.

The forward kernel is **fixed**. There is no θ in q . You do not have to simulate every hop. The marginal $q(x_t | x_0)$ is a closed-form Gaussian that interpolates from data to noise, so you can **jump** to any t in one shot.

In code that jump is one line: sample $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$ and mix it with x_0 using $\bar{\alpha}_t$. Training (note **12.2**) samples a random t , draws ε , forms x_t in one shot, and asks a net to guess ε . You never need to walk $1, 2, \dots, t$ unless you want a picture of the path.

2. Why we use it

GANs fight a min-max. Diffusion trains a denoiser with MSE. The kernel is not learned, so there is no discriminator to race. The training signal at each t is a denoising regression against a known ε .

Coverage is not automatic, but you are not playing an adversarial game. That is the trade this week is for. You still have to pick $\{\beta_t\}$ and T . Sampling will be a reverse chain (note **12.2**), which is slower than one GAN forward pass.

Likelihood-related bounds exist for this family in a way they do not for a vanilla GAN. You do not need the ELBO on the board today. You need: the forward process is a fixed noising schedule, and that is why the loss can be ordinary regression.

3. Architecture

The architecture of this note is a **Markov chain**, not a U-Net. (The net arrives in note **12.2**.) A variance schedule β_t or, equivalently, $\bar{\alpha}_t$ falling from near 1 to near 0, defines

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s.$$

The Markov step is $q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, \beta_t \mathbf{I})$. The marginal is closed form, so training never has to unroll the chain.



$\bar{\alpha}_t$ starts near 1 (the point is still itself) and falls toward 0 (the point is almost ε). Linear, cosine, and learned β_t are all ways to spend more steps where structure dies. Too fast a schedule and early t is already junk; too slow and you waste capacity on near-copies of x_0 .

T (often 1000 in image papers) is a discretization. Lab 12 uses a small T in 2-D so you can plot the path. Print $\bar{\alpha}_1, \bar{\alpha}_{T/2}, \bar{\alpha}_T$: they should fall.

4. How it works, step by step

The whole point of this note is that you can skip the chain at train time.

1. Choose a schedule $\{\beta_t\}$ and precompute $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$.
2. Take a clean sample x_0 (Lab 12: a 2-D point).
3. Draw a timestep t (training will draw it at random) and a noise $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$.
4. Form $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$ in one shot. Do not loop $s = 1, \dots, t$.
5. Hold x_t and ε for the reverse note: the net will be asked to guess ε .

If $\bar{\alpha}_t = 1$, then $x_t = x_0$. If $\bar{\alpha}_t = 0$, then $x_t = \varepsilon$. Mid-range $\bar{\alpha}_t$ is a mix: more noise than signal once $\bar{\alpha}_t$ is no longer close to 1. Same arithmetic in Lab 12 with a vector $x_0 = (1.5, -0.8)$ and a vector ε . The square roots are scalar; they multiply every coordinate.

5. Mathematical formulas

The one-step kernel and the closed-form marginal are

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, \beta_t \mathbf{I}),$$

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) \mathbf{I}).$$

The reparameterization you type in code is

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I}).$$

That one-liner is why the loss never walks 1, ..., t .

6. Positive points and negative points

Positive.

- The forward kernel is fixed, so the training target is a known ε rather than a moving adversary.
- You can jump to any t with one Gaussian draw, which makes training a single MSE later.
- The schedule is inspectable: print $\bar{\alpha}_1, \bar{\alpha}_{T/2}, \bar{\alpha}_T$ and watch them fall.
- The same algebra runs in Lab 12's 2-D points and in image DDPMs.

Negative.

- The forward process is the easy half. Reverse sampling (note **12.2**) is slow: you pay many network calls.
- A too-fast $\{\beta_t\}$ crashes $\bar{\alpha}_t$ in a few steps and the net never sees a lightly noised x_0 .
- A too-slow schedule wastes capacity on near-copies of the data.
- This is not a GAN: you gave up one-pass sampling in exchange for a stable regression.

When not to. If you need a sample in one forward pass and you can live with coverage bugs, that is still a GAN (Week 11).

7. Teaching this note

About **35 minutes** at the board, then **~6 minutes** of video. First of three Week-12 notes.

- **0–12 min.** Markov step vs closed-form marginal. Write α_t and $\bar{\alpha}_t$.
 - **12–22 min.** The one-line reparameterization. Draw x_0 mixing with ε .
 - **22–33 min.** Worked scalar step. Change $\bar{\alpha}_t$ from 1 to 0 and watch x_t .
 - **Then** play Umar Jamil **7:35–13:00** (forward/reverse cartoon, then the math of q). Pause on the closed-form $q(x_t | x_0)$.
-

8. Worked example

Take a 1-D point $x_0 = 1.0$ and a draw $\varepsilon = 2.0$. Then

$$x_t = \sqrt{\tilde{\alpha}_t} x_0 + \sqrt{1 - \tilde{\alpha}_t} \varepsilon.$$

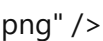
$\tilde{\alpha}_t$	$\sqrt{\tilde{\alpha}_t} x_0$	$\sqrt{1 - \tilde{\alpha}_t} \varepsilon$	x_t
1	1.0	0	1.0
0.64	0.8	$0.6 \times 2.0 = 1.2$	2.0
0	0	2.0	2.0

At $\tilde{\alpha}_t = 0.64$ you already sit at 2.0: more noise than signal. At $\tilde{\alpha}_t = 0$ you have thrown x_0 away; x_t is just ε .

Same arithmetic in Lab 12 with a vector $x_0 = (1.5, -0.8)$ and a vector ε . The square roots are scalar; they multiply every coordinate.

You can jump to any t without simulating $q(x_s | x_{s-1})$ for $s = 1, \dots, t$. That is why training is a single Gaussian draw, not a loop of length t .



$\tilde{\alpha}_t$ values on $x_0 = 1$, $\varepsilon = 2$ 

Pick a noise level, corrupt x , then learn to undo a bit. The closed-form jump is why the loss never walks $1, \dots, t$.

9. Where students get stuck

- Simulating all T Markov steps at train time. You sample t and use $q(x_t | x_0)$.
- Mixing up β_t , α_t , and $\tilde{\alpha}_t$. Write the three definitions every time.
- Setting every β_t huge. Then $\tilde{\alpha}_t$ crashes in a few steps and the net never sees a lightly noised x_0 .

10. Video

Watch [Umar Jamil, How diffusion models work](#), **7:35–13:00**.

Pause on the forward process q (no θ) and the closed-form jump to x_t . Skip the U-Net coding (after ~16:30) in this class. Note **12.2** continues the same video at the training/sampling chapters.

11. Practice

1. If $\tilde{\alpha}_t = 1$, what is x_t ? If $\tilde{\alpha}_t = 0$, what is x_t ?
2. Why can you sample x_t in one line instead of looping t steps of $q(x_s | x_{s-1})$?
3. A classmate sets every $\beta_t = 0.9$. What goes wrong in the first few t ?
4. $x_0 = 4$, $\tilde{\alpha}_t = 0.25$, $\varepsilon = -1$. Compute x_t . ($\sqrt{0.25} = 0.5$, $\sqrt{0.75} \approx 0.866$.)

5. $\beta_1 = 0.1, \beta_2 = 0.2$. Compute $\alpha_1, \alpha_2, \tilde{\alpha}_2$.